

TASK 3

Part 01

Question 1: What is the difference between `int.Parse` and `Convert.ToInt32` when handling null inputs?

- `int.Parse(null)`: Throws an `ArgumentNullException`. It strictly requires a non-null string value.
- `Convert.ToInt32(null)`: Handles null gracefully by returning 0 without throwing an exception.

Question 2: Why is `TryParse` recommended over `Parse` in user-facing applications?

- **Exception Overhead:** `Parse` throws exceptions (such as `FormatException` or `OverflowException`) on invalid input, which is expensive in terms of runtime performance.
- **Safe Handling:** `TryParse` returns a boolean indicating success or failure and uses an out parameter for the result, preventing application crashes.

Question 3: Explain the real purpose of the `GetHashCode()` method.

- **Hash-Based Lookup:** Provides an integer value used by hashtable-based collections (e.g., `Dictionary`, `HashSet`) to organize and group objects into buckets.
- **Fast Filtering:** Enables $O(1)$ performance by quickly testing if two objects might be equal before running a full `Equals()` check.

Question 4: What is the significance of reference equality in .NET?

- **Shared Memory Address:** Reference equality means two object variables store the same memory address on the Heap.
- **Shared State:** Modifying object properties through one variable directly updates the instance seen by any other variable referencing it.

Question 5: Why is string immutable in C#?

- **Thread Safety:** String instances can be shared across multiple threads without synchronization locks.
- **Security:** Ensures critical data like file paths and connection strings cannot be modified after validation.
- **String Interning:** Allows the CLR to optimize memory by storing only one instance of identical literal strings in an internal pool.
- **Dictionary Keys:** Guarantees that key hash codes remain consistent throughout execution.

Question 6: How does `StringBuilder` address the inefficiencies of string concatenation?

Standard string concatenation creates a new string object on the heap for each operation. `StringBuilder` maintains an internal dynamic buffer, allowing modifications to happen in-place without constant reallocations.

Question 7: Why is `StringBuilder` faster for large-scale string modifications?

It pre-allocates memory in chunks rather than reallocating memory for every change. This reduces time complexity from $O(N^2)$ down to $O(N)$ during repeated text appends in loops.

Question 8: Which string formatting method is most used and why?

String Interpolation (`$"Sum is {a} + {b} = {a + b}"`) is the most common approach because it improves code readability, supports compile-time checking, and reduces index matching errors compared to composite formatting.

Question 9: Explain how **StringBuilder** is designed to handle frequent modifications.

- **Mutable Buffer:** Wraps an internal `char [ ]` array.
- **In-Place Editing:** Operations like `Append`, `Insert`, and `Replace` modify array elements directly.
- **Automatic Resizing:** Doubles capacity when full to minimize array reallocation frequency.

Part 02

Question 10: Explain **String Immutability** and its underlying behavior in **C#**.

When a string variable is concatenated (e.g., `str += " text"`), C# does not modify the original memory. Instead, a new string instance is created on the heap with the combined value, and the variable reference is updated to point to the new location. The old instance becomes eligible for Garbage Collection.

Question 11: What is an **Enum** in **C#**, when is it used, and what are three built-in examples in **.NET**?

An enum is a value type that defines a set of named integer constants. It is used when a variable can only hold one of a predefined set of options.

Three built-in .NET enums:

- `System.ConsoleColor`
- `System.DayOfWeek`
- `System.IO.FileMode`

Question 12: Compare string and **StringBuilder** scenarios.

Feature / Requirement	string	StringBuilder
Modification Frequency	Few or static text operations	Frequent or dynamic modifications
Loop Concatenation	Not recommended (High GC overhead)	Recommended (Low GC overhead)
Memory Allocation	New instance per change	In-place internal buffer modification
Common Use Cases	UI text, config values, fixed formats	Dynamic XML/JSON creation, loggers

Part 03

Question 13: What is a **User-Defined Constructor** and what is its role in initialization?

A user-defined constructor is a class method with the same name as the class and no return type. It initializes fields and properties when an object is instantiated, enforces required parameters, and overrides the default parameterless constructor.

Question 14: Compare **Array** and **Linked List** data structures.

Property	Array	Linked List
Memory Layout	Contiguous memory allocation	Non-contiguous nodes linked by references
Index Access	O(1) Direct access	O(N) Traversal required
Insertion / Deletion	O(N) Shift operations required	O(1) Pointer updates
Sizing	Fixed length at initialization	Dynamic size
Memory Overhead	Minimal (stores data only)	Higher (stores data + node references)

Question 15: What are Jump Statements in C# and what are their primary types?

Jump statements unconditionally transfer control to another point in the program. The primary jump statements in C# include:

- **break:** Exits the nearest enclosing loop or switch statement.
- **continue:** Passes control to the next iteration of the enclosing loop.
- **return:** Exits the current method and optionally returns a value.
- **goto:** Transfers execution to a labeled statement (such as inside a switch case).
- **throw:** Signals the occurrence of an exception during program execution.

Question 16: How has the switch statement evolved across C# versions?

- **C# 1.0 - 6.0:** Basic pattern matching restricted to primitive constants, enums, and string values using traditional case blocks with mandatory **break/goto** statements.
- **C# 7.0:** Introduced Type Pattern Matching and **when** clauses, allowing evaluation of runtime object types and conditions.
- **C# 8.0:** Introduced Switch Expressions (**x switch { ... }**) providing a concise, functional syntax that returns values directly without repeated **case** or **break** keywords.
- **C# 9.0+:** Added Relational (**>**, **<=**), Logical (**and**, **or**, **not**), and Property patterns for complex conditional matching.